

Student Name	Balaji MJ
Register Number	TU-6253210211004
Project Title	Payment Transaction Management System
Week	Week 5
Subject Area	Database Management Systems (DBMS)

WEEK-5 PROJECT

Task Name:

Payment Transaction Management System

Description:

This Week-5 DBMS task focuses on designing and implementing a Payment Transaction Management System for an E-Commerce database. The objective is to maintain payment information in a structured relational table, store payment mode, date, amount, and transaction status, and use SQL queries to manage and analyse payment records.

The project should demonstrate how a database can record successful and failed payment transactions, retrieve transaction information, analyse the payment methods used by customers, and generate basic payment transaction reports.

1. PROJECT OBJECTIVE

The main objective is to create a reliable database component for storing and managing payment transactions. The system should allow administrators to identify transaction outcomes, review payment methods, and prepare summary reports.

2. CORE REQUIREMENTS

- Create a Payment table.
- Store payment details such as payment mode, payment date, amount, and transaction status.
- Manage successful and failed transactions.
- Analyse payment methods used by customers.
- Generate payment transaction reports.

Example: A customer places an order and pays ₹2,499 using UPI. The transaction is recorded with the payment date, payment mode, amount, and a status such as Successful or Failed.

3. DATABASE DESIGN

The Payment table should be connected to the customer and order information in the E-Commerce database. A payment record should have a unique transaction identifier and references to the relevant customer and order.

Column	Suggested Data Type	Purpose
payment_id	INT / VARCHAR	Unique payment transaction ID
order_id	INT / VARCHAR	Associated order
customer_id	INT / VARCHAR	Customer who made the payment
payment_mode	VARCHAR	UPI, Card, Net Banking, Wallet, COD, etc.
payment_date	DATE / DATETIME	Date and time of payment
amount	DECIMAL	Payment amount
payment_status	VARCHAR	Successful, Failed, Pending, Refunded

Sample Payment records

Payment ID	Order ID	Customer	Mode	Amount	Status
PAY101	ORD101	C101	UPI	2499	Successful
PAY102	ORD102	C102	Card	3499	Successful
PAY103	ORD103	C103	UPI	1299	Failed
PAY104	ORD104	C104	Wallet	1999	Pending
PAY105	ORD105	C105	Card	4999	Successful

4. CREATE PAYMENT TABLE

The first implementation step is to create the Payment table with suitable keys and data types. A primary key uniquely identifies each payment transaction.

```
CREATE TABLE Payment (  
    payment_id INT PRIMARY KEY,  
    order_id INT,  
    customer_id INT,  
    payment_mode VARCHAR(30),  
    payment_date DATETIME,  
    amount DECIMAL(10,2),  
    payment_status VARCHAR(20)  
);
```

Optional status validation

If the selected DBMS supports CHECK constraints, payment status can be restricted to known values.

```
CHECK (payment_status IN  
( 'Successful', 'Failed', 'Pending', 'Refunded' ))
```

5. STORE PAYMENT DETAILS

Example successful payment:

```
INSERT INTO Payment  
(payment_id, order_id, customer_id, payment_mode,  
 payment_date, amount, payment_status)  
VALUES  
(101, 1001, 1, 'UPI',  
 '2026-09-20 10:30:00', 2499.00, 'Successful');
```

Example failed payment:

```
INSERT INTO Payment  
(payment_id, order_id, customer_id, payment_mode,  
 payment_date, amount, payment_status)  
VALUES  
(102, 1002, 2, 'Card',  
 '2026-09-20 11:15:00', 3499.00, 'Failed');
```

Payment modes

- UPI
- Credit / Debit Card
- Net Banking
- Mobile Wallet
- Cash on Delivery (where applicable)

6. RETRIEVE PAYMENT TRANSACTION DETAILS

SELECT queries can be used to retrieve individual payment records or groups of transactions for business analysis.

Retrieve all payment records

```
SELECT *  
FROM Payment;
```

Retrieve selected payment details

```
SELECT payment_id, order_id, payment_mode,  
       amount, payment_status  
FROM Payment;
```

Find successful transactions

```
SELECT payment_id, order_id, amount, payment_mode  
FROM Payment  
WHERE payment_status = 'Successful';
```

Find failed transactions

```
SELECT payment_id, order_id, amount, payment_mode  
FROM Payment  
WHERE payment_status = 'Failed';
```

Find transactions using UPI

```
SELECT payment_id, customer_id, amount, payment_status  
FROM Payment  
WHERE payment_mode = 'UPI';
```

Sort payments by date

```
SELECT payment_id, payment_date, amount, payment_status  
FROM Payment  
ORDER BY payment_date DESC;
```

7. MANAGE SUCCESSFUL AND FAILED TRANSACTIONS

Transaction status allows the system to separate completed payments from unsuccessful attempts. This information can be used for payment monitoring and customer-support workflows.

Count successful transactions

```
SELECT COUNT(*) AS successful_transactions
FROM Payment
WHERE payment_status = 'Successful';
```

Count failed transactions

```
SELECT COUNT(*) AS failed_transactions
FROM Payment
WHERE payment_status = 'Failed';
```

Calculate successful transaction value

```
SELECT SUM(amount) AS successful_payment_value
FROM Payment
WHERE payment_status = 'Successful';
```

Find pending transactions

```
SELECT payment_id, order_id, amount, payment_date
FROM Payment
WHERE payment_status = 'Pending';
```

Payment status summary

```
SELECT payment_status,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount
FROM Payment
GROUP BY payment_status;
```

Business use

- Monitor failed payment attempts.
- Identify pending transactions.
- Measure the value of successful transactions.
- Prepare daily or monthly transaction summaries.

8. ANALYSE PAYMENT METHODS USED BY CUSTOMERS

Aggregate SQL functions can be used to understand which payment methods customers use most frequently and the transaction value associated with each method.

Count transactions by payment method

```
SELECT payment_mode,  
       COUNT(*) AS transaction_count  
FROM Payment  
GROUP BY payment_mode  
ORDER BY transaction_count DESC;
```

Total transaction value by payment method

```
SELECT payment_mode,  
       SUM(amount) AS total_amount  
FROM Payment  
GROUP BY payment_mode  
ORDER BY total_amount DESC;
```

Average transaction amount by method

```
SELECT payment_mode,  
       AVG(amount) AS average_amount  
FROM Payment  
GROUP BY payment_mode;
```

Successful payments by method

```
SELECT payment_mode,  
       COUNT(*) AS successful_count  
FROM Payment  
WHERE payment_status = 'Successful'  
GROUP BY payment_mode  
ORDER BY successful_count DESC;
```

Example payment-method report

Payment Mode	Transactions	Successful	Total Amount
UPI	120	108	■2,85,000
Card	95	89	■3,42,500
Wallet	55	49	■1,08,000
Net Banking	30	27	■1,21,000

The report values above are illustrative. Final values must be calculated from the actual project database.

9. PAYMENT TRANSACTION REPORTS

The Payment table can be used to generate basic reports for administrators and business analysts.

Daily payment report

```
SELECT payment_date,
       payment_mode,
       payment_status,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount
FROM Payment
GROUP BY payment_date, payment_mode, payment_status
ORDER BY payment_date DESC;
```

Successful transaction report

```
SELECT payment_mode,
       COUNT(*) AS successful_transactions,
       SUM(amount) AS successful_amount
FROM Payment
WHERE payment_status = 'Successful'
GROUP BY payment_mode
ORDER BY successful_amount DESC;
```

Failed transaction report

```
SELECT payment_mode,
       COUNT(*) AS failed_transactions,
       SUM(amount) AS failed_amount
FROM Payment
WHERE payment_status = 'Failed'
GROUP BY payment_mode
ORDER BY failed_transactions DESC;
```

Overall payment summary

```
SELECT
  COUNT(*) AS total_transactions,
  SUM(amount) AS total_value,
  AVG(amount) AS average_transaction
FROM Payment;
```

10. END-TO-END WORKFLOW

Customer / Order Information
↓
Create Payment Table
↓
Store Payment Details
↓
Record Payment Status
↓
Retrieve Transactions
↓
Analyse Payment Methods
↓
Generate Payment Reports

11. VALIDATION AND TESTING

The project should verify that payment records are stored correctly and that SQL reports return the expected results.

- Verify that every payment has a unique payment_id.
- Check that payment amounts are positive where required.
- Verify that payment_status contains only valid status values.
- Check that payment_mode is stored consistently.
- Verify successful and failed transaction counts.
- Compare aggregate totals with the underlying transaction records.
- Test sorting by payment date and transaction amount.

Example test cases

Test	Expected behaviour
payment_status = 'Successful'	Only successful transactions
payment_status = 'Failed'	Only failed transactions
payment_mode = 'UPI'	Only UPI transactions
GROUP BY payment_mode	One summary per payment method
SUM(amount)	Correct total payment value
COUNT(*)	Correct transaction count

12. RECOMMENDED TECHNOLOGIES

- MySQL, PostgreSQL, Oracle SQL, or SQLite.
- MySQL Workbench, pgAdmin, Oracle SQL Developer, or another SQL client.
- Relational database tables for customers, orders, and payments.
- Optional application frontend for displaying payment history and reports.

13. FINAL PROJECT DELIVERABLES

1. Payment table design with appropriate columns and keys.
2. Sample payment transaction records.
3. SQL queries for inserting payment details.
4. Queries for retrieving payment transactions.
5. Successful and failed transaction management queries.
6. Payment-method analysis using GROUP BY and aggregate functions.
7. Transaction counts and total payment-value reports.
8. Daily or period-based payment reports.
9. Validation and testing results.
10. Screenshots or exported SQL query results.

14. PROJECT OBJECTIVE

The main objective of this Week-5 DBMS project is to implement a Payment Transaction Management System that can store and manage transaction information in a relational database. The project demonstrates how SQL can be used to retrieve payment records, distinguish successful and failed transactions, analyse payment methods, and generate basic reports.

15. FINAL OUTPUT

Output	Description
Payment Details	Transaction ID, order, customer, mode, date, amount, and status
Successful Transactions	Transactions completed successfully
Failed Transactions	Transactions that did not complete successfully
Payment Method Analysis	Usage and value by payment method
Transaction Reports	Counts, totals, averages, and status summaries

Note: Sample records and report values in this document are illustrative. Final transaction counts, totals, and payment-method statistics should be generated from the actual database.